

簡介：透過 Workflows 自動化調度管理無伺服器工作

來源：<https://codelabs.developers.google.com/codelabs/cloud-workflows-intro?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將瞭解如何使用 Workflows 自動化調度管理及自動化 Google Cloud 及以 HTTP 為基礎的 API 服務。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [工作流程總覽](#)
4. [部署第一個 Cloud Function](#)
5. [部署第二個 Cloud Function](#)
6. [連結兩個 Cloud Functions](#)
7. [連結外部 HTTP API](#)
8. [部署 Cloud Run 服務](#)
9. [連結 Cloud Run 服務](#)
10. [恭喜！](#)

1. 簡介



您可以使用 [Workflows](#) 建立無伺服器工作流程，按照您定義的順序，將一系列無伺服器工作串連在一起。您可以結合 Google Cloud API、Cloud Functions 和 Cloud Run 等無伺服器產品，並呼叫外部 API，藉此建構彈性的無伺服器應用程式。

這項產品不需管理基礎架構，並能視需求順暢調度資源，甚至將資源縮減至零。採用「以量計價，即付即用」的定價模式，因此您只需要為執行時間付費。

在本程式碼研究室中，您將瞭解如何透過 Workflows 連結各種 Google Cloud 服務和外部 HTTP API。具體來說，您會將兩個公開 Cloud Functions 服務、一個私有 Cloud Run 服務和一個外部公開 HTTP API 連結到工作流程。

課程內容

- Workflows 基本概念。
 - 如何將公開 Cloud Functions 連線至 Workflows。
 - 如何將私有 Cloud Run 服務連結至 Workflows。
 - 如何透過 Workflows 連線至外部 HTTP API。
-

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 G Suite 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：console.cloud.google.com。

The screenshot shows the Google Cloud Platform interface for creating a new project. The top navigation bar includes the Google Cloud Platform logo and a 'Select a project' dropdown. Below this, the 'Select a project' section features a search bar and a 'NEW PROJECT' button. The 'New Project' form has three main sections: 'Project name *' with the value 'my-kubernetes-codelab', 'Project ID: my-kubernetes-codelab. It cannot be changed later. EDIT', and 'Location *' with the value 'No organization'. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 G Suite 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

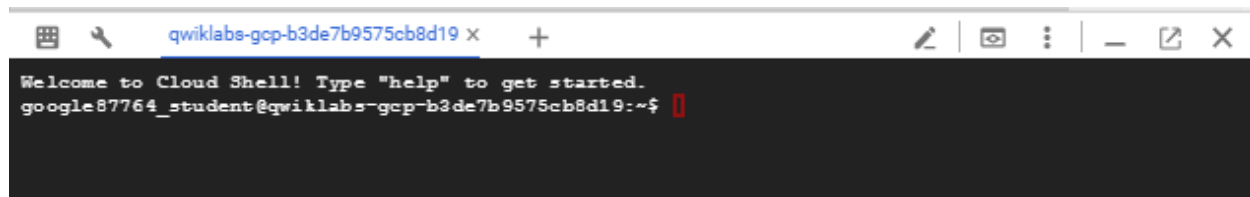
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 GCP 主控台，按一下右上角工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell interface. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a black background with white text. The text reads: 'Welcome to Cloud Shell! Type "help" to get started.' followed by the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor. The browser window includes standard navigation buttons and a tab titled 'qwiklabs-gcp-b3de7b9575cb8d19'.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本實驗室的所有工作都可在瀏覽器上完成。

步驟 3 · 約 2 分鐘

3. 工作流程總覽

基本資訊

工作流程是由一系列步驟組成，這些步驟使用以 YAML 為基礎的 Workflows 語法描述。這是工作流程的定義。如需 Workflows YAML 語法的詳細說明，請參閱「[語法參考資料](#)」頁面。

建立工作流程後，系統會部署工作流程，讓工作流程準備就緒，隨時可執行。執行作業是指單次執行工作流程定義中包含的邏輯。所有工作流程執行作業都是獨立的，而且產品支援同時執行大量作業。

啟用服務

在本程式碼研究室中，您將連結 Cloud Functions、Cloud Run 服務和 Workflows。您也會在建構服務時使用 Cloud Build 和 Cloud Storage。

啟用所有必要服務：

```
gcloud services enable \  
  cloudfunctions.googleapis.com \  
  run.googleapis.com \  
  workflows.googleapis.com \  
  cloudbuild.googleapis.com \  
  storage.googleapis.com
```

在下一個步驟中，您會在工作流程中連結兩個 Cloud Functions。

步驟 4 · 約 6 分鐘

4. 部署第一個 Cloud Function

第一個函式是以 Python 撰寫的隨機數字產生器。

建立並前往函式程式碼的目錄：

```
mkdir ~/randomgen
cd ~/randomgen
```

在目錄中建立 `main.py` 檔案，並加入以下內容：

```
import random, json
from flask import jsonify

def randomgen(request):
    randomNum = random.randint(1,100)
    output = {"random":randomNum}
    return jsonify(output)
```

收到 HTTP 要求後，這項函式會產生介於 1 到 100 之間的隨機數字，然後以 JSON 格式傳回呼叫端。

函式會依賴 Flask 處理 HTTP，因此我們需要將其新增為依附元件。Python 中的依附元件由 pip 代管，並以名為 `requirements.txt` 的中繼資料檔案表示。

在同一個目錄中建立 `requirements.txt` 檔案，並加入以下內容：

```
flask>=1.0.2
```

使用 HTTP 觸發條件部署函式，並允許未經驗證的要求：

```
gcloud functions deploy randomgen \
  --runtime python312 \
  --trigger-http \
  --allow-unauthenticated
```

函式部署完畢後，您可以在控制台顯示的 `url` 屬性下方，或使用 `gcloud functions describe` 指令查看函式網址。

您也可以使用下列 `curl` 指令，前往函式的網址：

```
curl $(gcloud functions describe randomgen --format='value(url)')
```

函式已可供工作流程使用。

步驟 5 · 約 6 分鐘

5. 部署第二個 Cloud Function

第二個功能是乘數。將收到的輸入值乘以 2。

建立並前往函式程式碼的目錄：

```
mkdir ~/multiply
cd ~/multiply
```

在目錄中建立 `main.py` 檔案，並加入以下內容：

```
import random, json
from flask import jsonify

def multiply(request):
    request_json = request.get_json()
    output = {"multiplied": 2*request_json['input']}
    return jsonify(output)
```

收到 HTTP 要求後，這項函式會從 JSON 主體中擷取 `input`，然後將其乘以 2，並以 JSON 格式傳回呼叫端。

在同一個目錄中建立相同的 `requirements.txt` 檔案，並加入以下內容：

```
flask>=1.0.2
```

使用 HTTP 觸發條件部署函式，並允許未經驗證的要求：

```
gcloud functions deploy multiply \
  --runtime python312 \
  --trigger-http \
  --allow-unauthenticated
```

部署函式後，您也可以使用下列 `curl` 指令前往函式的網址：

```
curl $(gcloud functions describe multiply --format='value(url)') \
-X POST \
-H "content-type: application/json" \
-d '{"input": 5}'
```

函式已可供工作流程使用。

6. 連結兩個 Cloud Functions

在第一個工作流程中，將這兩個函式連結在一起。

使用以下內容建立 `workflow.yaml` 檔案。

請務必將網址值替換為函式的實際網址，並使用正確的區域和專案 ID

```
- randomgenFunction:
  call: http.get
  args:
    url: https://<region>-<project-id>.cloudfunctions.net/randomgen
  result: randomgenResult
- multiplyFunction:
  call: http.post
  args:
    url: https://<region>-<project-id>.cloudfunctions.net/multiply
    body:
      input: ${randomgenResult.body.random}
  result: multiplyResult
- returnResult:
  return: ${multiplyResult}
```

在這項工作流程中，您會從第一個函式取得隨機數字，並將該數字傳遞至第二個函式。結果是相乘後的隨機數字。

部署第一個工作流程：

```
gcloud workflows deploy workflow --source=workflow.yaml
```

執行第一個工作流程：

```
gcloud workflows execute workflow
```

工作流程執行完畢後，您可以傳入上一步中提供的執行作業 ID，查看結果：

```
gcloud workflows executions describe <your-execution-id> --workflow workflow
```

輸出內容會包含 `result` 和 `state`：

```
result: '{"body":{"multiplied":108},"code":200 ... }
```

```
...
```

```
state: SUCCEEDED
```

步驟 7 · 約 6 分鐘

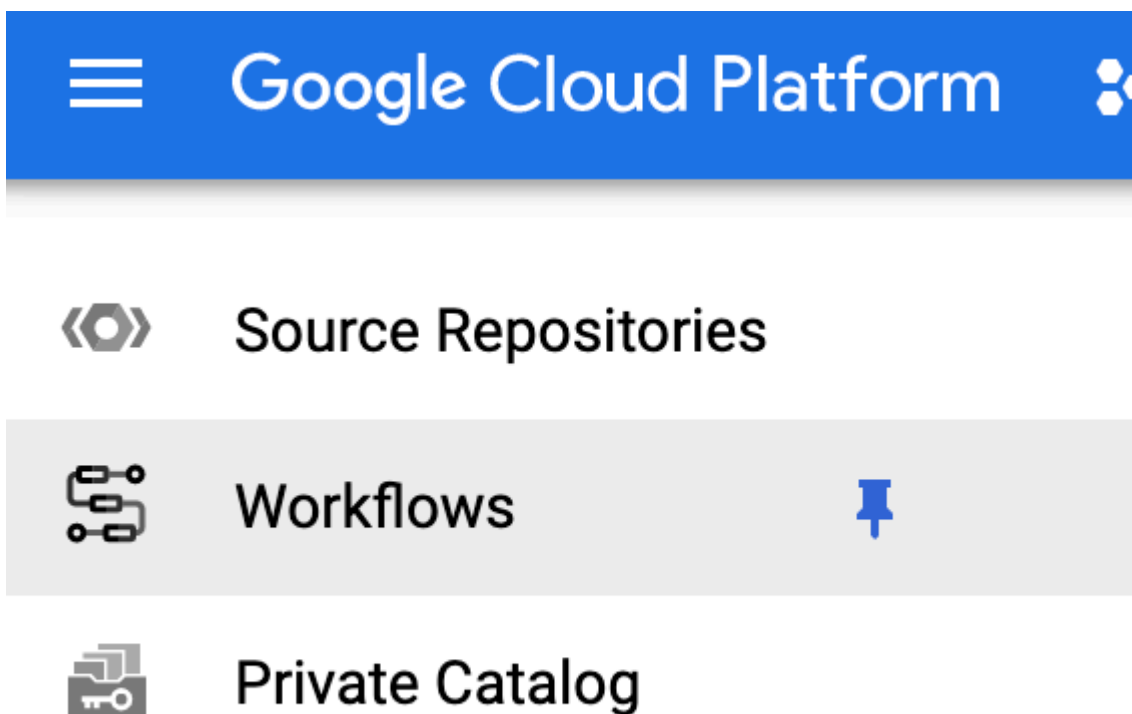
7. 連結外部 HTTP API

接著，您會在工作流程中將 `math.js` 連線為外部服務。

在 `math.js` 中，您可以評估數學運算式，如下所示：

```
curl https://api.mathjs.org/v4/?'expr=log(56)'
```

這次您要使用 Cloud Console 更新工作流程。在 Google Cloud 控制台中找出 `Workflows`：



找出工作流程，然後按一下「`Definition`」分頁標籤：

Workflows BETA

[Workflow Details](#) [EDIT](#) [EXECUTE](#) [DELETE](#)

workflow

EXECUTIONS

LOGS

DEFINITION

Name	workflow
Last deployed revision	000001-c35
Deployment time	Sep 8, 2020, 9:35:57 AM
Description	None
Region	us-central1
Service account	projects/workflows-atamel/serviceAccounts/workflow-sa@workflows-atamel.iam.gserviceaccount.com
Labels	None

Code

```
- randomgenFunction:
  call: http.get
  args:
    url: https://us-central1-workflows-atamel.cloudfunctions.net/randomgen
  result: randomgenResult
- multiplyFunction:
  call: http.post
  args:
    url: https://us-central1-workflows-atamel.cloudfunctions.net/multiply
    body:
      input: ${randomgenResult.body.random}
  result: multiplyResult
- returnResult:
  return: ${multiplyResult}
```

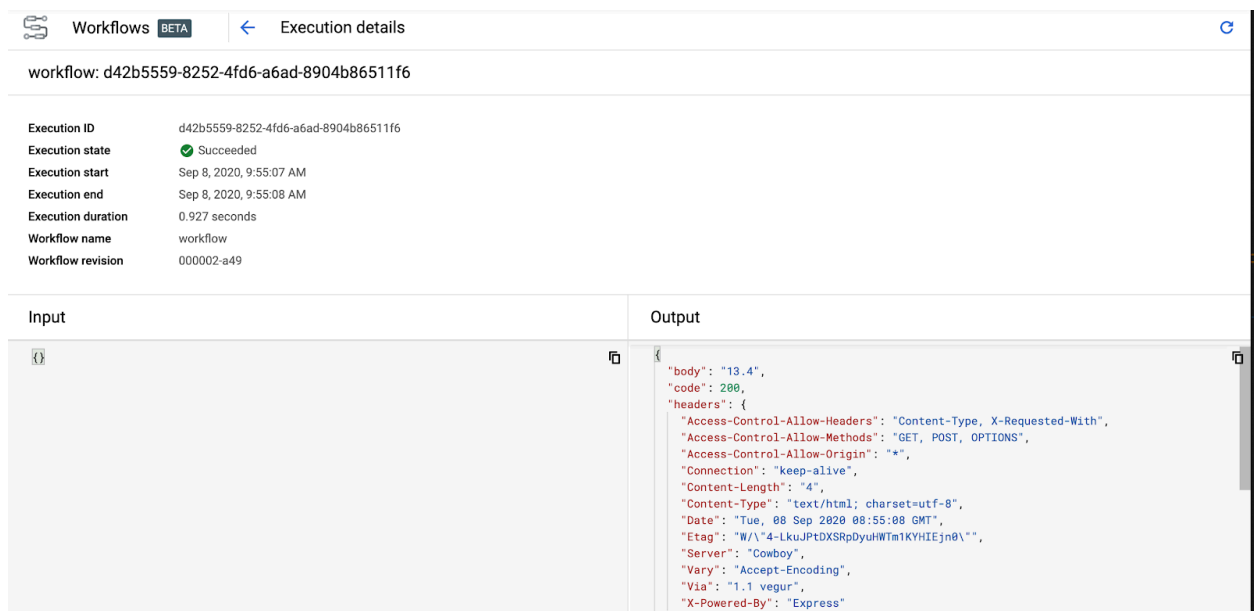
編輯工作流程定義，並加入對 `math.js` 的呼叫。

請務必將網址值替換為函式的實際網址，並使用正確的區域和專案 ID

```
- randomgenFunction:
  call: http.get
  args:
    url: https://<region>-<project-id>.cloudfunctions.net/randomgen
  result: randomgenResult
- multiplyFunction:
  call: http.post
  args:
    url: https://<region>-<project-id>.cloudfunctions.net/multiply
    body:
      input: ${randomgenResult.body.random}
  result: multiplyResult
- logFunction:
  call: http.get
  args:
    url: https://api.mathjs.org/v4/
    query:
      expr: ${"log(" + string(multiplyResult.body.multiplied) + ")"}
  result: logResult
- returnResult:
  return: ${logResult}
```

工作流程現在會將 multiply 函式的輸出內容，饋送至 math.js 中的記錄函式呼叫。

使用者介面會引導您編輯及部署工作流程。部署完成後，按一下 **Execute** 即可執行工作流程。您會看到執行作業的詳細資料：



Workflows **BETA** Execution details

workflow: d42b5559-8252-4fd6-a6ad-8904b86511f6

Execution ID	d42b5559-8252-4fd6-a6ad-8904b86511f6
Execution state	✔ Succeeded
Execution start	Sep 8, 2020, 9:55:07 AM
Execution end	Sep 8, 2020, 9:55:08 AM
Execution duration	0.927 seconds
Workflow name	workflow
Workflow revision	000002-a49

Input	Output
	<pre>{ "body": "13.4", "code": 200, "headers": { "Access-Control-Allow-Headers": "Content-Type, X-Requested-With", "Access-Control-Allow-Methods": "GET, POST, OPTIONS", "Access-Control-Allow-Origin": "*", "Connection": "keep-alive", "Content-Length": "4", "Content-Type": "text/html; charset=utf-8", "Date": "Tue, 08 Sep 2020 08:55:08 GMT", "Etag": "W/\"4-LkuJPtDXSRpDyuhWTm1KYHIEjn0\"", "Server": "Cowboy", "Vary": "Accept-Encoding", "Via": "1.1 vegur", "X-Powered-By": "Express" } }</pre>

請注意狀態碼 **200** 和 **body**，其中包含記錄函式的輸出內容。

您剛才將外部服務整合到工作流程中，真是太棒了！

步驟 8 · 約 6 分鐘

8. 部署 Cloud Run 服務

最後一個部分是呼叫私有 Cloud Run 服務，完成工作流程。也就是說，工作流程必須通過驗證，才能呼叫 Cloud Run 服務。

Cloud Run 服務會傳回傳入數字的 `math.floor`。

建立並前往服務程式碼的目錄：

```
mkdir ~/floor
cd ~/floor
```

在目錄中建立 `app.py` 檔案，並加入以下內容：

```
import json
import logging
import os
import math

from flask import Flask, request

app = Flask(__name__)

@app.route('/', methods=['POST'])
def handle_post():
    content = json.loads(request.data)
    input = float(content['input'])
    return f"{math.floor(input)}", 200

if __name__ != '__main__':
    # Redirect Flask logs to Gunicorn logs
    gunicorn_logger = logging.getLogger('gunicorn.error')
    app.logger.handlers = gunicorn_logger.handlers
    app.logger.setLevel(gunicorn_logger.level)
    app.logger.info('Service started...')
else:
    app.run(debug=True, host='0.0.0.0', port=int(os.environ.get('PORT', 8080)))
```

Cloud Run 會部署容器，因此您需要 `Dockerfile`，且容器必須繫結至 `0.0.0.0` 和 `PORT` 環境變數，因此需要上述程式碼。

收到 HTTP 要求後，這項函式會從 JSON 主體中擷取 `input`，然後呼叫 `math.floor` 並將結果傳回給呼叫端。

在同一個目錄中建立下列 `Dockerfile`：

```
# Use an official lightweight Python image.
# https://hub.docker.com/_/python
FROM python:3.7-slim

# Install production dependencies.
RUN pip install Flask gunicorn

# Copy local code to the container image.
WORKDIR /app
COPY . .

# Run the web service on container startup. Here we use the gunicorn
# webserver, with one worker process and 8 threads.
# For environments with multiple CPU cores, increase the number of workers
# to be equal to the cores available.
CMD exec gunicorn --bind 0.0.0.0:8080 --workers 1 --threads 8 app:app
```

建構容器：

```
export SERVICE_NAME=floor
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/${SERVICE_NAME}
```

容器建構完成後，請部署至 Cloud Run。請注意 `no-allow-unauthenticated` 旗標。這樣可確保服務只接受經過驗證的呼叫：

```
gcloud run deploy ${SERVICE_NAME} \
  --image gcr.io/${GOOGLE_CLOUD_PROJECT}/${SERVICE_NAME} \
  --platform managed \
  --no-allow-unauthenticated
```

部署完成後，服務即可用於工作流程。

步驟 9 · 約 4 分鐘

9. 連結 Cloud Run 服務

如要設定 Workflows 呼叫私有 Cloud Run 服務，請先建立供 Workflows 使用的服務帳戶：

```
export SERVICE_ACCOUNT=workflows-sa
gcloud iam service-accounts create ${SERVICE_ACCOUNT}
```

將 `run.invoker` 角色授予服務帳戶。這樣一來，服務帳戶就能呼叫已通過驗證的 Cloud Run 服務：

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \
  --member
  "serviceAccount:${SERVICE_ACCOUNT}@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \
  --role "roles/run.invoker"
```

更新 `workflow.yaml` 中的工作流程定義，納入 Cloud Run 服務。請注意，您也加入了 `auth` 欄位，確保 Workflows 在呼叫 Cloud Run 服務時會傳遞驗證符記：

請務必將網址值替換為函式的實際網址，並使用正確的區域和專案 ID


```

- randomgenFunction:
  call: http.get
  args:
    url: https://<region>-<project-id>.cloudfunctions.net/randomgen
  result: randomgenResult
- multiplyFunction:
  call: http.post
  args:
    url: https://<region>-<project-id>.cloudfunctions.net/multiply
    body:
      input: ${randomgenResult.body.random}
  result: multiplyResult
- logFunction:
  call: http.get
  args:
    url: https://api.mathjs.org/v4/
    query:
      expr: ${"log(" + string(multiplyResult.body.multiplied) + ")"}
  result: logResult
- floorFunction:
  call: http.post
  args:
    url: https://floor-<random-hash>.run.app
    auth:
      type: OIDC
    body:
      input: ${logResult.body}
  result: floorResult
- returnResult:
  return: ${floorResult}

```

更新工作流程。這次傳入服務帳戶：

```

gcloud workflows deploy workflow \
  --source=workflow.yaml \
  --service-
account=${SERVICE_ACCOUNT}@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com

```

執行工作流程：

```
gcloud workflows execute workflow
```

幾秒後，您就能查看工作流程執行作業，瞭解結果：

```
gcloud workflows executions describe <your-execution-id> --workflow workflow
```

輸出內容會包含整數 `result` 和 `state`：

```
result: '{"body":"5","code":200 ... }
```

```
...
```

```
state: SUCCEEDED
```

步驟 10 · 約 1 分鐘

10. 恭喜！

恭喜您完成本程式碼研究室。

涵蓋內容

- Workflows 基本概念。
 - 如何將公開 Cloud Functions 連線至 Workflows。
 - 如何將私有 Cloud Run 服務連結至 Workflows。
 - 如何透過 Workflows 連線至外部 HTTP API。
-